

Vision Transformer Implementation

DATA SETUP:

- Normalized Data with
- `cifar10_mean = [0.4914, 0.4822, 0.4465]`
- `cifar10_std = [0.2023, 0.1994, 0.2010]`
- Random Cropping, Random Horizontal Flip and Random Augmentation Applied to data
- Batch Size 128

MODEL LAYERS :

1. Patch Embedding:

- Patch Embedding, creates patches and returns Tensor of Dimension: 128 X 65 X 128: 128 images 64 tokens (patches + CLS) per image, 128 dimensions per token (128-dimension vector per patch)
- Using Convolution 2d as Projection Layer as it's more efficient than a Linear Layer. Kernel and Stride are assigned to patch size as we want patches and not overlaps. For square images number of patches is: **(image size/patch size) ^2** (CIFAR10 is square images)
- The CLS token (classification token) in Vision Transformers is a special learnable parameter that serves as a placeholder for aggregating global information from all image patches for classification tasks
- Positional Embedding is optional for Convolutional Transformer Architecture hence may or may not be added to the Tensor; this is because Inductive Bias added later will cater to relationships between patches.
- Tensor Output from Patch Embedding Operation: [128, 65, 384] : 128 images 65 tokens per image (CLS token + 64 Patches), 128-dimension representation of each token
- Kernel Size = Stride = Patch Size = 4

Patch Embedding Block:

```
PatchEmbedding( (projection): Conv2d(3, 384, kernel_size=(4, 4), stride=(4, 4)))
```

2. Transformer Encoder:

- Each head processes Embedding Size/ Number of Heads Dimensions
- Pre Norm Architecture (Norm before adding to Multi Head Attention as per VIT Paper) stabilizes gradients during training, enables training of deeper networks also
- Normalizing before each major operation to prevent gradient Explosion and Vanishing Gradients during Back Prop
- Q K V for Multihead Self Attention come from Convolutional layers unlike VIT
- Groups Specified for Convolutional Layer is the most important, Groups is equal to the Embedding Dimension, this results in a convolution run on each channel with a 3X3 Kernel, capturing information about immediate Neighbours.
- This process is also known as **Local Inductive Bias**.
- For Computing Convolutions: **Stride size for K and V is set to 2** for efficiency reasons as it cuts computation cost by 4X with negligible accuracy loss.
- **Stride Size for Q is set to 1** to preserve the full spatial resolution of the output.
- Tensors are converted to Spatial Format for Convolutions to run.
- CLS Token is removed from the Tensors, Convolution Run to get Q K V values,
- Q K V are transformed back to Sequence format, then CLS Token is added back to the Tensor.
- Residual Connection: Adds Original Input to the output from Multi Head Attention
- Output is passed through the MLP described below
- 12 Transformer Blocks Used

Transformer Block:

```
ConvTransformerEncoder(  
    (norm1): LayerNorm((384,)), eps=1e-05, elementwise_affine=True)  
    (conv_q): Conv2d(384, 384, kernel_size=(3, 3), stride=(1, 1),  
padding=(1, 1), groups=384)  
    (conv_k): Conv2d(384, 384, kernel_size=(3, 3), stride=(2, 2),  
padding=(1, 1), groups=384)  
    (conv_v): Conv2d(384, 384, kernel_size=(3, 3), stride=(2, 2),  
padding=(1, 1), groups=384)  
    (multiHeadAttn): MultiheadAttention(  
    (out_proj): NonDynamicallyQuantizableLinear(in_features=384,  
out_features=384, bias=True)  
    )  
    (norm2): LayerNorm((384,)), eps=1e-05,  
elementwise_affine=True)  
    (mlp): MLP(  
        (fc1): Linear(in_features=384, out_features=512, bias=True)  
        (fc2): Linear(in_features=512, out_features=384, bias=True)  
        (dropout): Dropout(p=0.1, inplace=False)  
    )  
)
```

3. MLP Layer:

- Position-wise feed-forward network MLP: Applies Non-Linear Transformation on the token generated from Multi Head Attention
- MLP applies the same transformation independently to each of the 65 tokens (64 Patches + CLS Token)
- Performs an EXPAND ACTIVATE PROJECT BACK operation, Maintains original Dimensions
- Gaussian Error Linear Unit Activation Function with DROPOUT_RATE=0.1

4. Linear Classification Head:

- Linear Classification Head performs logits= $W \cdot x + b$, Output is logits of num classes size vector (10 for Cifar10)
- CLS Token is Extracted and passed to the Linear Classification Head.

Classification Head:

```
Linear(in_features=384, out_features=10, bias=True)
```

Training Setup:

- Loss Function is Cross Entropy Loss
- AdamW Optimizer with LEARNING_RATE = $3e-4$, betas= (0.9, 0.999), weight_decay=0.01
- Cosine Annealing Learning Rate Scheduler

Results:

1. CIFAR10

After 50 Epochs: 88% Accuracy in half the number of Epochs as Vision Transformer.

- Train Loss: 0.3326
- Train Accuracy: 0.8835
- Test Accuracy: 0.8793
- Precision: 0.88
- Recall: 0.88
- F1 Score: 0.88



